MTServe Technical Report

Ning Cheng
Wuhan University

Xin Wang
Wuhan University

Chi Ma
Meituan

Pu Wang
Meituan

Menglei Zhou
Meituan

Junyi Qiu
NVIDIA

Bin Chai
NVIDIA

Qiaorui Chen
NVIDIA

Jiayu Sun
NVIDIA

Shijie Liu
NVIDIA

Zehuan Wang
NVIDIA

Lei Yu
Meituan

Chuan Liu
Meituan

Fei Jiang
Meituan

Wei Lin
Meituan

Bo Du
Wuhan University

Jiawei Jiang
Wuhan University

Xiao Yan
Wuhan University

Abstract

Generative recommendation (GR) offers superior modeling capabilities but suffers from prohibitive inference costs due to the repeated encoding of long user histories. While cross-request Key-Value (KV) cache reuse presents a significant optimization opportunity, the massive scale of individual user states causes a storage explosion that far exceeds physical GPU limits. Unlike standard LLM workloads that benefit from highly shared prefixes, GR serving demands massive, individualized state restorations. This unique access pattern exposes existing engines to severe cache churn and prohibitive I/O stalls. To address this, we propose MTServe, a hierarchical cache management system that virtualizes GPU memory using host RAM as a scalable backup. To bridge the I/O gap between tiers, MTServe introduces a suite of system-level optimizations, including a dual-granularity (Page-Chunk) storage layout, a proactive cache offloading mechanism, and an asynchronous multi-stream pipeline for layer-wise compute-transfer overlapping. On both public and production datasets, MTServe delivers up to a 3.5× speedup while maintaining near-perfect hit ratios (> 98%).

PVLDB Reference Format:
Ning Cheng, Xin Wang, Chi Ma, Pu Wang, Menglei Zhou, Junyi Qiu, Bin Chai, Qiaorui Chen, Jiayu Sun, Shijie Liu, Zehuan Wang, Lei Yu, Chuan Liu, Fei Jiang, Wei Lin, Bo Du, Jiawei Jiang, and Xiao Yan. MTServe Technical Report. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 Introduction

For years, recommendation systems have been dominated by traditional deep learning models, such as Wide & Deep [3] and Deep Interest Networks [41], which primarily focus on discriminative ranking through point-wise scoring. However, inspired by the transformative success of Large Language Models (LLMs) in capturing complex patterns, the industry is rapidly pivoting toward the paradigm of generative recommendation (GR) [1, 7, 13, 15, 33, 37]. By framing user–item interaction modeling as a sequence transduction problem, GR models leverage Transformer-based architectures to unlock high-order dependencies within extensive interaction histories that were previously under-explored. While major industrial players are aggressively pursuing this paradigm to achieve unprecedented personalization, the transition has encountered a formidable obstacle: the prohibitive computational cost of processing long-term histories at scale. Consequently, satisfying the strict latency requirements of online services often demands an unsustainable amount of hardware resources, making serving infrastructure optimization a critical priority for large-scale deployment.

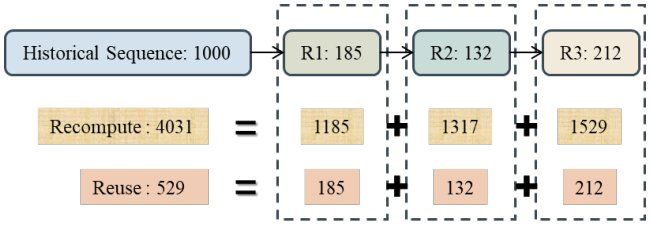

Figure 1: Comparison of total tokens processed across three consecutive requests from the same user: *Recompute* (4,031 tokens) vs. *Reuse* (529 tokens).

Cache Reuse. The inference workload in GR is characterized by processing a user's interaction sequence, which typically comprises an extensive historical prefix followed by a relatively small segment of new interactions. As illustrated in Figure 1, the incremental portion appended during each visit is often a tiny fraction of the total sequence length. However, standard inference engines often treat each request as an isolated event, leading to massive computational redundancy. To mitigate this, *user-side Key–Value (KV) caching* persists the intermediate activations—specifically the Key and Value tensors—of a specific user's historical prefix across multiple independent visits. By retrieving these user-level persistent states, the system can bypass the expensive re-encoding of the invariant history and focus on the incremental updates. As shown in Figure 1, for three consecutive requests (R_1, R_2, R_3) from the same user, the

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

recompute paradigm is forced to repeatedly encode the cumulative prefix, totaling *4,031 tokens*. In contrast, the user-side cache reuse workload is significantly reduced to merely *529 tokens*.

Challenges. To store these massive user states without exhausting expensive GPU memory, adopting a hierarchical GPU–CPU tiered caching architecture is a necessary choice. However, directly migrating LLM-centric serving frameworks (e.g., vLLM[17], TensorRT-LLM[23]) to generative recommendation fails to deliver the expected performance. This failure stems from a divergence in workload characteristics. First, LLM serving thrives on cross-request prompt sharing, whereas GR workloads exhibit individualized histories with *near-zero prefix sharing* across users. Second, LLM interactions are typically localized within continuous, short-lived chat sessions, while GR demands *long-term state durability* across intermittent user visits spanning days. Finally, LLM inference is bottlenecked by a lengthy, compute-heavy autoregressive decoding phase; in contrast, GR inference is overwhelmingly *prefill-dominant* and utilizes shallower models. This fundamental shift yields a substantially higher *I/O-to-computation ratio* for GR compared to LLMs, featuring significantly heavier PCIe data movement.

When standard hierarchical engines[39] encounter these distinct GR properties, they face three severe system-level challenges:
① **Single-Granularity Bottlenecks.** Standard LLM engines manage memory at a fine-grained level (e.g., paging) to minimize VRAM fragmentation. However, under the massive state swapping required by GR's individualized histories, transferring gigabytes of data via discrete, small pages generates excessive PCIe transaction overhead. A single granularity fails to balance memory allocation flexibility and I/O transmission efficiency.
② **Reactive Eviction Stalls.** Traditional frameworks typically employ reactive eviction—moving states to host memory only when VRAM is exhausted. Given the continuous cache churn driven by massive user bases and intermittent visits in GR, this reactive policy inevitably triggers sudden, massive forced data transfers directly on the critical path. Waiting for these evictions causes severe synchronization stalls that inflate inference latency.
③ **Rigid Execution Pipelines.** Standard engines enforce rigid execution barriers, waiting for historical data transfers to complete before initiating computation. Because GR is prefill-dominant and highly I/O-intensive, this serialized paradigm fails to hide the massive PCIe latency. Consequently, the GPU is severely underutilized, and the pipeline is paralyzed by I/O wait times.

Our Solution MTServe. To address these GR-specific challenges and break the GPU memory wall, we propose MTServe, a hierarchical KV cache management system tailored for serving generative recommendation in Meituan. Developed in collaboration with NVIDIA, MTServe operates as an individualized virtual memory system that dynamically tiers user states between a fast GPU cache and a scalable host memory backup.

To implement this tiered architecture efficiently and overcome the limitations of LLM engines, MTServe incorporates three co-designed system optimizations: First, we introduce a *dual-granularity storage abstraction* (Page–Chunk) to reconcile the conflict between computational flexibility and transmission efficiency. While fine-grained GPU paging reduces memory fragmentation, coarse-grained CPU chunking maximizes PCIe bandwidth utilization during massive state swapping. Second, to achieve non-blocking cache reclamation, we introduce a *proactive, chunk-granularity offloading policy*. Instead of reactively offloading scattered pages upon VRAM exhaustion, MTServe continuously streams accumulated user states to host RAM in full chunks via background threads, enabling instantaneous, zero-copy metadata evictions on the critical path. Finally, we design an *asynchronous multi-stream execution pipeline* integrated with a custom *double buffering mechanism*. While double buffering fundamentally accelerates the raw PCIe data transfer throughput, the multi-stream architecture breaks rigid wait-then-compute barriers by enabling layer-wise compute-transfer overlapping, which parallelizes the historical KV data onload for the upcoming transformer layer with the attention computation of the current layer.

We evaluate MTServe on the KuaiRand-1K and a production MT dataset. By fundamentally resolving the coupled eviction and serialized execution overheads of standard engines, MTServe achieves up to a **3.5**× latency speedup over the recompute baseline. Crucially, it maintains a near-perfect total hit ratio (>**98%**) even when the active working set far exceeds physical GPU memory capacity.

In summary, our main contributions are as follows:

- We identify the significant opportunity for user-side KV cache reuse in generative recommendation. Furthermore, we explicitly characterize the fundamental workload divergence from LLM serving— zero prefix sharing, intermittent durability requirements, and prefill-dominant I/O—that renders standard hierarchical caching frameworks inadequate.
- We propose MTServe, a hierarchical serving system designed as an individualized virtual memory. It breaks the GPU memory wall through a scalable two-tier (GPU-CPU) cache hierarchy, unlocking massive capacity while being specifically architected for the high I/O-to-computation ratio of GR serving.
- We introduce a suite of co-designed system mechanisms to ensure highly efficient execution, featuring a *Page–Chunk storage layout* for balanced I/O, a *proactive cache offloading policy* with strict capacity backpressure, and an *asynchronous multi-stream pipeline* for layer-wise compute-transfer overlapping.

2 Preliminaries

Generative Recommendation (GR). GR frames the ranking task as a sequence transduction problem. Given a user interaction sequence $\mathbf{H}_u$ comprising P historical tokens and a candidate set $C = \{c_1, c_2, \ldots, c_N\}$, the model estimates the relevance score γ_i for each candidate conditioned on the historical context. As shown in Figure 2, the input sequence $\mathbf{x}$ is constructed by concatenating the history and candidates:

$$\mathbf{x} = [\mathbf{x}_{1:P}; \mathbf{x}_{P+1:P+N}] \tag{1}$$

where $\mathbf{x}_{1:P} = \mathbf{H}_u$ and $\mathbf{x}_{P+1:P+N} = C$, with a total length of $T = P + N$.

The architecture consists of L stacked blocks. An initial embedding layer projects each token x_t into a d-dimensional vector $\mathbf{e}_t^{(0)}$. Following the state-of-the-art HSTU architecture [37], each stacked block $\Psi^{(l)}$ $(1 \le l \le L)$ functions similarly to a standard Transformer layer, centering around an attention-based transformation.

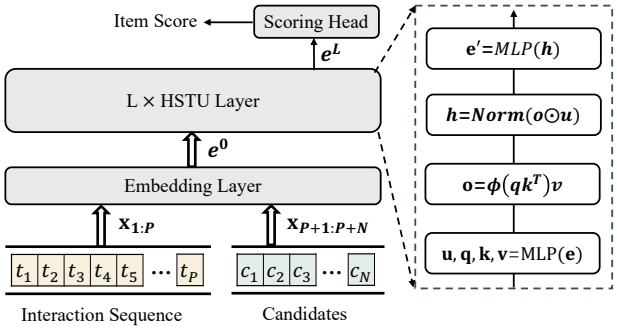

Figure 2: The architecture of HSTU model for GR [37].

The internal computation of layer l is formulated as:

$$\mathbf{u}^{(l)}, \mathbf{q}^{(l)}, \mathbf{k}^{(l)}, \mathbf{v}^{(l)} = \text{Split}(\phi_1(\text{Linear}(\mathbf{e}_{1:T}^{(l-1)}))) \tag{2}$$

$$\mathbf{o}^{(l)} = \phi_2(\text{Attention}(\mathbf{q}^{(l)}, \mathbf{k}^{(l)}, \mathbf{v}^{(l)})) \tag{3}$$

$$\mathbf{e}_{1:T}^{(l)} = \text{MLP}(\text{Norm}(\mathbf{o}^{(l)} \odot \mathbf{u}^{(l)})) \tag{4}$$

where ϕ_1 and ϕ_2 denote the SiLU activation functions. The hidden representations undergo L such transformations to produce the final contextualized embeddings.

Focusing on the terminal position T, the representation $\mathbf{e}_T^{(L)}$ is fed into a *scoring head* to obtain the ranking logits $\mathbf{y} = \mathbf{W}_{out}\mathbf{e}_T^{(L)}$. For any candidate c_i associated with its unique identifier token $\omega_i \in \mathcal{V}$, its relevance is directly determined by the corresponding logit value $y[\omega_i]$. The final ranking result is derived by sorting the candidates $\{c_1, \ldots, c_N\}$ in descending order of their logit values.

Serving GR Models with KV Cache. To accelerate serving, we decouple the user history into a cached prefix $\mathbf{H}_{pre}$ (length P_{pre}) and incremental tokens $\Delta\mathbf{H}$ (length ΔP). When a request arrives, the system retrieves the pre-computed Key-Value tensors $(\mathbf{K}_{pre}^{(l)}, \mathbf{V}_{pre}^{(l)})$ from the cache. The model then computes only the new projections $(\mathbf{k}_\Delta^{(l)}, \mathbf{v}_\Delta^{(l)})$ for the incremental input $\mathbf{x}_\Delta = [\Delta\mathbf{H}; C]$. The complete attention context at layer l is seamlessly assembled by concatenating the cached and newly generated states:

$$\mathbf{K}_{total}^{(l)} = [\mathbf{K}_{pre}^{(l)}; \mathbf{k}_\Delta^{(l)}], \quad \mathbf{V}_{total}^{(l)} = [\mathbf{V}_{pre}^{(l)}; \mathbf{v}_\Delta^{(l)}] \tag{5}$$

This *user-side caching* paradigm isolates inference latency from the total history length, reducing the attention complexity from $O(T^2)$ to $O((T - P_{pre}) \cdot T)$. Finally, only the KV states of $\Delta\mathbf{H}$ are appended to the cache to facilitate future reuse.

3 MTServe Overview

We design MTServe to tackle the GPU memory bottleneck in generative recommendation, which stems from the dual challenges of a massive user base and long interaction histories. Functioning as a plug-and-play cache management module, MTServe integrates seamlessly with attention-based generative models to enable efficient user-side KV cache reuse. Figure 3 illustrates the overall architecture and the seven-step inference workflow.

3.1 System Components

MTServe comprises four primary components that collaborate to execute high-throughput inference by efficiently managing and utilizing the hierarchical KV cache.

- **Generative Recommendation (GR) Model.** The core inference engine (e.g., HSTU [37]). It interacts with the logical cache interface provided by the KV cache manager, remaining agnostic to the underlying physical storage and tiering logic.
- **KVCacheManager.** The central controller residing on the host. It handles metadata management, including sequence length tracking, LRU replacement, and page table maintenance. The page table acts as a logical-to-physical mapping structure where each entry tracks the physical location of a fixed-length segment (e.g., a 32-token page) of a user's KV cache, serving as the atomic unit for GPU memory management.
- **Hierarchical Cache Store.** A dual-tier storage backend (as shown in the center of Figure 3). It consists of the *CPU Tier* (Chunked Store) as a scalable backup store and the *GPU Tier* (Paged Store) as the primary cache for low-latency access. Both tiers are managed via a paging/chunking mechanism to minimize internal fragmentation.
- **Data Transfer Path.** A dedicated pipeline for moving KV blocks between tiers. It utilizes *Pinned Memory Buffers* on the CPU to enable high-speed DMA and *Onload/Offload Buffers* on the GPU for data staging. This path supports asynchronous, double-buffered transfers to overlap I/O with computation.

Memory Layout. To eliminate dynamic allocation overheads, the memory inside the *GPU Tier* is logically partitioned into two distinct functional regions (illustrated within the cyan box in Figure 3). (1) *Device Page Store (Primary Cache):* A discrete, non-contiguous paged storage area that holds the persistent KV tensors of active users for low-latency retrieval. (2) *Onload Buffer:* A pre-allocated contiguous VRAM region serving a dual purpose: it acts as an active inference area where the model instantly accesses newly fetched history without blocking, and simultaneously functions as a staging ground for the scatter kernel to transform data layouts.

3.2 Inference Workflow

The lifecycle of an inference request in MTServe follows a streamlined seven-step workflow, as depicted by the circled numbers in Figure 3. In our serving scenario, each incoming request encapsulates a unique user identifier, a sequence of incremental interaction tokens newly arrived since the user's last visit, and a set of candidate items to be ranked.

❶ **Metadata Preparation.** Upon receiving a batch of requests, the *KVCacheManager* prepares execution metadata by retrieving cached sequence lengths, resolving page mappings, and allocating new GPU pages. If device memory is exhausted, the LRU policy identifies victim pages for zero-copy eviction.

❷ **Data Onload.** For requests requiring historical data from the host (i.e., a GPU cache miss), the system asynchronously transfers relevant KV blocks from the *CPU Tier* into a pinned memory buffer, and then via DMA to the GPU's *Onload Buffer*, naturally overlapping with input preprocessing.

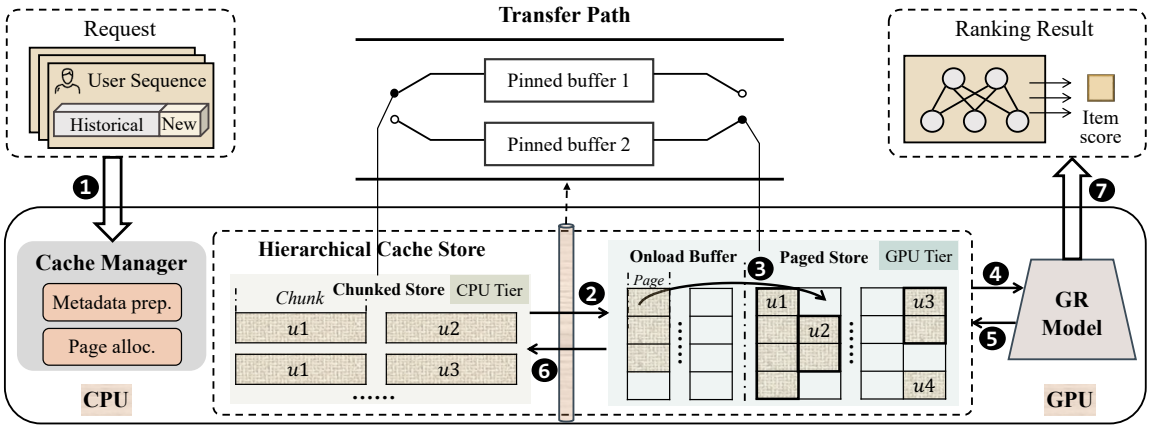

Figure 3: The overall architecture and seven-step inference workflow of MTServe.

❸ **Cache Scattering.** Once the data resides in the Onload Buffer, the system executes an intra-GPU transformation. A dedicated scatter kernel redistributes the contiguous data from the Onload Buffer into pre-allocated discrete pages within the *GPU Tier*.

❹ **Model Inference.** With the cache populated, the GR Model executes the forward pass. It utilizes the page table metadata to retrieve historical KV pairs from the Device Page Store and combines them with current inputs for self-attention.

❺ **Cache Update.** The new KV states generated during the inference step are appended directly to the allocated pages in the Device Page Store, updating the user's history in real time.

❻ **Cache Offload.** A background process manages the write-back of new KV states. When a user's accumulated new tokens reach the predefined host chunk size, the corresponding GPU pages are gathered into a transiently allocated *Offload Buffer*. This buffer is then asynchronously transferred to the *CPU Tier*, ensuring the CPU backend maintains a persistent record of the user history without stalling the critical path.

❼ **Ranking Result.** Finally, the hidden representations are passed to the scoring head to calculate ranking scores.

4 Key Designs

This section details the system optimizations of MTServe, focusing on the synergy between its hierarchical storage abstraction, asynchronous data pipeline, and non-blocking cache management policy. Together, these designs maximize hardware utilization and ensure low-latency serving even under extreme memory pressure.

4.1 Dual-Granularity Hierarchical Storage

Hybrid Storage Granularity. Current LLM inference engines typically rely on single-granularity page management (e.g., PagedAttention [17]) to optimize memory allocation. However, directly applying this uniform fine granularity to GR workloads triggers catastrophic I/O bottlenecks. Because GR serving entails massive, individualized state restorations, moving data across the PCIe bus using small, discrete pages incurs excessive transaction overheads (as detailed in Appendix D.3). Fundamentally, while fine-grained paging minimizes VRAM fragmentation, it fails to saturate PCIe bandwidth during bulk data swapping. To resolve this conflict, MTServe introduces a *dual-granularity storage layout*, tailoring memory management to the distinct hardware characteristics of the GPU and CPU tiers (illustrated in the center of Figure 3).

- **GPU Tier (Fine-grained Paging):** On the GPU, where capacity is the primary bottleneck, we adopt a fine-grained page management strategy (e.g., 32 tokens per page). This granularity reduces internal fragmentation, allowing the cache to expand dynamically with the growing sequence length. The GPU-resident *Paged Store* is defined as a pre-allocated tensor of shape $[L, N_{\text{pages}}, 2, S_{\text{page}}, H, D]$, where L denotes layer numbers, N_{pages} total capacity, S_{page} page size, and H, D the attention head dimensions. By treating GPU memory as a pool of discrete pages, any free slot can be allocated to any user, effectively eliminating external fragmentation and improving HBM utilization.
- **CPU Tier (Coarse-grained Chunking):** Conversely, in host RAM, where capacity is abundant but bandwidth is constrained by the PCIe interface, we manage data in coarser *Chunks* (e.g., 1024 tokens). Since PCIe throughput is highly sensitive to transaction overhead, transferring a single large chunk is significantly more efficient than moving multiple small, discontinuous pages. This design amortizes the startup cost of DMA transactions, ensuring that the I/O subsystem can saturate the physical link during bulk data movement.

To bridge these granularities, MTServe employs a robust logical-to-physical indexing mechanism that maps continuous user sequences to physically scattered pages and chunks. A unified page mapping is shared across all layers, while each layer maintains its own KV cache storage and independent transfer synchronization, enabling the layer-wise compute-transfer overlapping. We defer the detailed translation logic to Appendix D.1.

4.2 Efficient Data Movement

Efficiently moving KV blocks between host and device is critical for maintaining high system throughput, especially given the high I/O-to-computation ratio of recommendation models. We optimize this process by employing dedicated scatter/gather kernels for GPU

memory layout transformation, reinforced by a double-buffered pipeline designed to saturate the physical PCIe bus bandwidth.

Pipelined Onload via Double Buffering. The onload process (Step ②, Figure 3) involves host-side memory preparation and PCIe DMA transfer. To hide this I/O latency, MTServe employs a fine-grained *double-buffering strategy* utilizing two pinned memory buffers (*Pinned Buffer 1 & 2* in Figure 3). Crucially, we strictly align the size of these buffers with the host storage *Chunk* granularity. This alignment transforms host-side preparation into a high-speed sequential block copy, explicitly bypassing the heavy CPU overhead of gathering non-contiguous memory segments. Operating in a *ping-pong* fashion, the CPU sequentially populates Buffer 1 while the DMA engine concurrently transfers the previously prepared Buffer 2 to the GPU. This overlapping ensures the DMA engine remains saturated. Finally, a custom-optimized *scatter kernel* (Step ③) distributes the contiguous data from the Onload Buffer into discrete physical pages within the GPU Tier.

Proactive Offload & Capacity Control. To enable non-blocking cache reclamation, MTServe discards reactive eviction in favor of a *proactive offloading* strategy. A background pipeline triggers exactly when a user's accumulated new tokens reach the pre-defined *host chunk size* (Step ⑥ of Figure 3). A gather kernel collects the dispersed GPU pages into a transient offload buffer, which is then asynchronously transferred and persisted to the CPU Tier. Because the host maintains an up-to-date backup, GPU pages can be instantaneously reclaimed under memory pressure via zero-copy metadata updates, avoiding severe synchronization stalls. Furthermore, to prevent GPU memory exhaustion from transient buffer accumulation, MTServe enforces a strict quota on the total volume of pending offload tasks. The system tracks the aggregated tokens currently queued in GPU transient buffers; if a new offload task would cause this total to exceed the pre-configured limit, the task is rejected via an admission control mechanism. This acts as a critical backpressure valve, ensuring that background maintenance tasks do not destabilize the primary inference service by consuming excessive VRAM, thereby guaranteeing system reliability even under extreme heavy traffic bursts.

4.3 Cache Management

Effective cache management hinges on exploiting the temporal patterns inherent in user traffic. As shown in Figure 4, the arrival intervals of consecutive requests from the same user exhibit a heavy-tailed distribution, where the majority of requests occur within a short temporal window. This strong temporal locality motivates the adoption of a Least-Recently-Used (LRU) policy, which prioritizes retaining recently active users in the limited GPU memory to maximize the cache hit ratio.

Allocation and Eviction Policy. To implement the LRU policy efficiently at scale, MTServe maintains a global doubly-linked list augmented with an auxiliary hash map to ensure $O(1)$ access and update complexity. Page allocation is tightly coupled with this policy: when the system requires pages for a new request or incremental token generation, it first attempts to draw from the free pool. If the pool is exhausted, the system reclaims pages from the tail of the LRU list, targeting the least recently active users. Crucially,

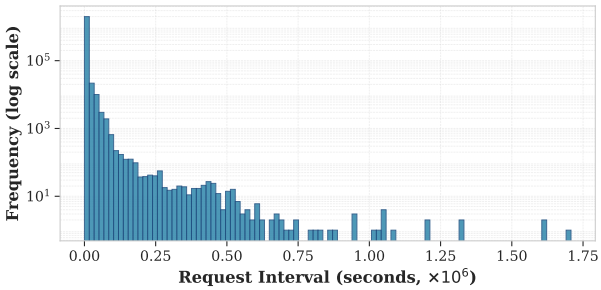

Figure 4: Distribution of arrival intervals between consecutive requests from the same user. The heavy-tailed distribution indicates that most users return within a short period, exhibiting strong temporal locality.

because the background offload process already persists data to the host (Section 4.2), this eviction is a *zero-copy metadata operation*. Victim pages are simply marked as free in the page table, entirely bypassing blocking device-to-host transfers and eliminating latency overhead on the critical inference path.

User Locking Protocol. To prevent race conditions during concurrent operations, we enforce a strict *User Locking* protocol. Users currently undergoing background offload (Step ⑥, Figure 3) are explicitly *skipped* by the LRU victim selection. This coordination prevents physical pages from being prematurely reclaimed or overwritten while still being read by the DMA engine, guaranteeing memory consistency under high-concurrency workloads.

4.4 Asynchronous Pipelining

We orchestrate the aforementioned components using a multi-stream execution model to maximize hardware utilization. The detailed end-to-end inference workflow and the corresponding pseudocode (Algorithm 1) are provided in Appendix C.

Four-Stream Concurrency. MTServe utilizes four independent, non-blocking CUDA streams to overlap disparate workloads, effectively functioning as a producer-consumer system: (1) *Compute Stream* for core model execution, including the Transformer-based forward pass (*Step 8*); (2) *Onload Stream* for host-to-device DMA transfers to retrieve historical KV chunks (initiated in *Step 2*); (3) *Scatter Stream* for intra-GPU data redistribution from the onload buffer to the paged store (initiated in *Step 7*); and (4) *Offload Stream* for background persistence by transferring new KV states back to the host (initiated in *Step 9*). Dependencies between these streams are managed via fine-grained CUDA events to ensure correct execution order without global stalls. This multi-stream design allows the system to preprocess metadata and prepare embeddings (*Steps 3–6*) in parallel with the heavy KV data movement occurring in the Onload Stream.

Fine-Grained Layer-wise Synchronization. A critical optimization in our design is the *implicit layer-wise synchronization* (Step 8.3). Instead of a coarse-grained barrier that waits for the entire batch's data to be loaded, we implement a staggered execution mechanism using `KVOnloadHandle`. Before executing the attention mechanism for Layer l, the Compute Stream waits specifically for

Table 1: End-to-end latency, speedup, and hit ratio comparison across batch sizes (BS) of 1, 4, and 8.

BS	Method	KuaiRand-1K				MT Dataset			
		Latency (ms)	Speedup	GPU Hit	Total Hit	Latency (ms)	Speedup	GPU Hit	Total Hit
1	Recomputation	21.2	1.00×	-	-	17.1	1.00×	-	-
	GPU-Only	17.4	1.22×	63.52%	63.52%	13.1	1.31×	61.02%	61.02%
	TRT-LLM	14.6	1.45×	63.52%	99.22%	11.6	1.47×	61.02%	99.23%
	LMCache	13.5	1.57×	63.52%	99.22%	11.2	1.53×	61.02%	99.23%
	MTServe	**11.7**	**1.81**×	63.52%	98.56%	**10.1**	**1.69**×	61.02%	98.04%
4	Recomputation	72.8	1.00×	-	-	55.7	1.00×	-	-
	GPU-Only	42.1	1.73×	63.92%	63.92%	32.8	1.70×	61.61%	61.61%
	TRT-LLM	40.6	1.79×	63.92%	99.22%	31.8	1.75×	61.61%	99.24%
	LMCache	35.2	2.07×	63.92%	99.22%	27.7	2.01×	61.61%	99.24%
	MTServe	**27.5**	**2.65**×	63.92%	98.57%	**23.9**	**2.33**×	61.61%	98.64%
8	Recomputation	143.6	1.00×	-	-	108.1	1.00×	-	-
	GPU-Only	72.9	1.97×	64.36%	64.36%	46.5	2.32×	61.48%	61.48%
	TRT-LLM	64.2	2.24×	64.36%	99.23%	46.0	2.35×	61.48%	99.24%
	LMCache	52.2	2.75×	64.36%	99.23%	37.1	2.91×	61.48%	99.24%
	MTServe	**40.2**	**3.57**×	64.36%	98.59%	**30.2**	**3.58**×	61.48%	98.65%

the completion event of Layer l's onload operation. This allows the computation of shallower layers to proceed immediately once their specific data is ready, even if the transfer for deeper layers is still in progress. By deeply overlapping the memory-bound data movement with the compute-bound attention mechanism, MTServe significantly reduces the end-to-end inference latency and mitigates the impact of the high I/O-to-computation ratio.

5 Experimental Evaluation

5.1 Experimental Settings

Model Architecture and Cache Configuration. We employ the Hierarchical Sequential Transduction Unit (HSTU) [37] as our backbone model. The default model configuration consists of 8 Transformer-based layers, 4 attention heads, and a hidden dimension of 128 per head, totaling a model size and complexity suitable for large-scale deployment. By default, MTServe is configured with a fine-grained page size of 32 tokens, a primary cache pool of 40,960 pages, and a coarse-grained chunk size of 1,024 tokens.

Baselines. To rigorously evaluate MTServe, we compare it against four representative serving paradigms, including two advanced LLM-serving frameworks:

- **Recomputation:** This paradigm, exemplified by standard implementations of *HSTU* [37] and *OneRec* [7], re-encodes the entire user interaction history for every inference request, serving as the performance lower bound.
- **GPU-Only Cache:** This baseline stores KV states exclusively in high-bandwidth GPU memory with a strict LRU eviction policy, like BAT [29] and OneTrans [38]. Once VRAM is saturated, the least recently active user sessions are permanently evicted.
- **TensorRT-LLM:** One of the most representative LLM inference frameworks in the industry. We select it as our representative baseline due to its exceptional performance and extensive, tailored optimizations for NVIDIA hardware [4]. To ensure a fair architectural comparison, we co-designed a hierarchical variant with NVIDIA engineers, explicitly augmenting it with our Page–Chunk layout and double buffering to mitigate its native I/O bottlenecks (detailed in Appendix D.3).
- **LMCache:** A KV cache management layer designed for enterprise LLM serving [21]. To isolate the benefits of our asynchronous scheduling, we similarly enhanced this baseline by integrating our double buffering mechanism (detailed in Appendix D.3).

Table 2: Latency decomposition (ms) of the inference pipeline on KuaiRand-1K at BS=8. The step labels correspond to the stages defined in Algorithm 1.

Stage	Recomp.	GPU-Only	TRT-LLM	LMCache	MTServe
1-2. Prep. Meta.	-	0.1	8.8	5.9	0.1
3. Strip Tokens	-	0.5	0.4	0.4	0.5
4. Embedding	9.7	1.5	1.1	1.1	1.1
5. Data Layout	5.7	3.6	3.1	3.1	3.1
6. Await Meta.	-	0.1	-	-	0.1
7. Update Meta.	-	0.05	0.02	0.02	0.01
8. HSTU Infer.	127.4	66.3	47.9	39.7	34.3
9. Offload KV	-	-	-	0.01	0.01
10. Postprocess	0.8	0.7	2.6	1.9	1.5

Evaluation Metrics. We evaluate the system using *Average Latency* (ms) and *Token Hit Ratio* (%), where the latter represents the percentage of historical tokens retrieved from the hierarchical cache relative to the total sequence length.

Datasets. We evaluate MTServe on two datasets: **KuaiRand-1K** [11] (a public streaming recommendation benchmark) and the **MT Dataset** (a production trace from Meituan featuring long interaction sequences). Detailed statistical information and preprocessing are deferred to Appendix A.

Hardware and Implementation. All experiments are conducted on an NVIDIA A100 GPU (80GB) server. Detailed specifications and low-level implementations are deferred to Appendix D.2.

5.2 End-to-End Latency Reduction

We evaluate the end-to-end performance of MTServe across different workloads. As shown in Table 1, MTServe consistently achieves the lowest latency across all batch sizes and datasets.

MTServe vs. Recomputation. On KuaiRand-1K, MTServe achieves a **1.81**× speedup at BS=1, which scales to an impressive **3.57**× at BS=8 (40.2 ms vs. 143.6 ms). The performance gain is similarly pronounced on the production MT dataset, where MTServe delivers a **3.58**× speedup at BS=8 (30.2 ms vs. 108.1 ms). The massive recompute latency on KuaiRand-1K dataset is driven by its extensive sequence lengths (up to 20,000 items), which create a computational bottleneck that slows down the entire batch.

MTServe vs. LLM Serving. Beyond traditional baselines, MTServe outperforms LLM engines like LMCache and TRT-LLM. Notably, on the MT dataset, the performance of TRT-LLM (46.0 ms at BS=8) drops to the level of the GPU-Only baseline (46.5 ms). This highlights a major architectural limitation: standard engines rely on rigid, synchronous I/O. When handling the massive KV states of GR workloads, this I/O process stalls, completely canceling out the benefits of tiered storage. To pinpoint where these bottlenecks occur and how MTServe resolves them, Table 2 breaks down the end-to-end pipeline execution.

Initially, during the preparation stage (Steps 1-2), standard LLM engines suffer from blocking memory evictions. Under the high QPS and rapid cache churn of GR, allocating new space forces the system to synchronously wait for old physical states to be written back to host memory. While LMCache incorporates partial proactive offloading, its reliance on synchronous eviction still causes a 5.9 ms stall at BS=8. TRT-LLM's purely passive eviction is even worse, stalling for 8.8 ms. MTServe avoids this bottleneck via *asynchronous proactive offloading*. Because physical user states are asynchronously flushed to the CPU in full chunks in the background, MTServe enables instantaneous, zero-copy metadata eviction on the critical path, drastically reducing this overhead.

Furthermore, MTServe accelerates the core execution phase (Step 8) by fundamentally restructuring the data movement paradigm. Standard engines enforce rigid execution barriers at varying granularities. TRT-LLM imposes a batch-level barrier, forcing the GPU to idle until the historical data for the entire batch is fully loaded. LMCache improves upon this with layer-wise prefetching, yet it still strictly serializes operations within each layer as *onload* → *scatter* → *inference*. These rigid pipelines inflate core inference times (39.7 ms for LMCache and 47.9 ms for TRT-LLM at BS=8). Conversely, MTServe breaks these barriers through a two-pronged approach. First, it employs *parallel onload initiation*, dispatching data transfer requests concurrently with input preprocessing (Steps 2–5) to absorb the initial I/O latency. Second, it introduces *flexible compute-transfer overlap* during the core computation. By configuring the onload buffer to serve directly as an active inference workspace, MTServe enables the concurrent execution of data scattering and attention computation. This transforms the pipeline into *onload* → *[scatter & inference]* paradigm. This architectural shift allows the memory loading to seamlessly overlap with the attention computation of preceding layers, hiding the data transfer overhead and pulling the core inference time down to 34.3 ms.

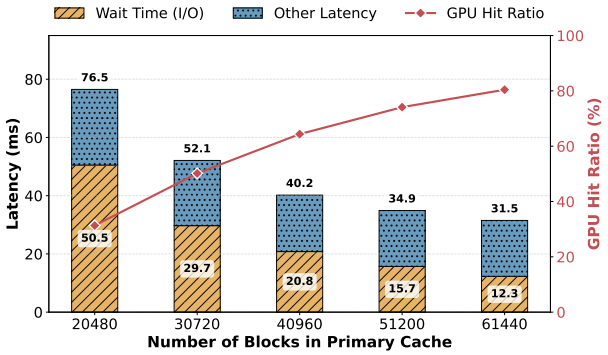

Figure 5: Impact of GPU Cache Store capacity on latency components and cache hit ratio.

5.3 Cache Efficiency

The performance advantage of MTServe is rooted in its ability to maintain a near-perfect hit ratio by breaking the GPU memory wall. As shown in Table 1, while the *GPU-Only* baseline is capped at a GPU hit ratio of approximately **63.5%–64.3%** on KuaiRand-1K due to strict VRAM limits, MTServe utilizes host RAM as a scalable backup store to achieve a total hit ratio of **98.56%–98.59%**.

Interestingly, the total hit ratio of MTServe is marginally lower than that of TRT-LLM and LMCache (which plateau around **99.2%**). This slight drop is an architectural trade-off governed by MTServe's *offload capacity control*, which actively rejects a small fraction of offload tasks to prevent VRAM exhaustion during traffic bursts. Crucially, while advanced LLM engines can achieve these peak theoretical hit ratios via hierarchical storage, they pay a massive latency penalty to maintain them due to their blocking I/O operations. By sacrificing less than 1% of the hit ratio to ensure a fully non-blocking, asynchronous retrieval process, MTServe accelerates the core attention mechanism without stalling the pipeline, validating its readiness for industrial-scale generative recommendation where active working sets far exceed physical device limits.

5.4 System Overhead Analysis

We examine the fine-grained execution costs of each pipeline stage to assess the system overhead introduced by MTServe. As shown in the latency decomposition (Table 2), the system maintains high efficiency by keeping control-plane operations strictly minimal.

- **Metadata and Control Overhead:** The combined CPU overhead for metadata preparation, page table resolution, and synchronization (*Steps 1-2, 6, and 7*) remains remarkably low, totaling less than 0.3 ms even at BS=8. This confirms that our chunk-based indexing and zero-copy eviction policies introduce a negligible computational burden.
- **Asynchronous Offload Efficiency:** The initiation of the background offload process (*Step 9*) consumes merely 0.01 ms. As designed in Section 4.2, the actual KV transfers are handled by dedicated background threads, ensuring that the persistence of user states imposes zero blocking latency on the inference path.

Table 3: Impact of Chunk Size on inference latency (ms).

Chunk Size	512	1024	2048	4096
Wait Time (I/O)	32.41	20.84	19.82	17.48
Comp Time (Kernel)	12.48	13.21	14.67	17.93
End2end Latency	52.4	40.2	42.6	43.5

- **I/O vs. Computation Trade-off:** MTServe explicitly trades expensive redundant encoding for highly optimized KV restoration. Shifting the workload to our layer-wise pipeline replaces the massive re-computation cost of 127.4 ms at BS=8 with a 34.3 ms inference step. The net saving of 93.1 ms conclusively proves that the I/O penalty of moving hierarchical data is entirely outweighed by the computational savings of attention reuse.

5.5 Sensitivity Analysis

Impact of GPU Cache Capacity. We evaluate system robustness by adjusting the *Primary Cache* capacity from 20,480 to 61,440 blocks. As shown in Figure 5, increasing the capacity significantly improves the *GPU Hit Ratio*, leading to a sharp decline in I/O wait time. However, this performance gain comes at the cost of a higher memory footprint as detailed in Table 5, the VRAM consumption scales from 10 GB to 30 GB across these settings. Notably, even at the most constrained setting (20,480 blocks, 10 GB VRAM), MTServe still achieves an end-to-end latency of **76.5 ms**, representing a **1.88**× speedup over the RE baseline (143.6 ms).

Impact of Chunk Size. We examine the impact of *Offload Chunk Size* on the trade-off between data movement and computation. To quantify this, we instrument the core HSTU inference loop (*Step 8*) to separately measure the synchronization overhead (*Wait Time*) and the kernel execution time (*Comp Time*).

As shown in Table 3, increasing the chunk size from 512 to 4096 tokens reduces the *Wait Time* (32.41 ms → 17.48 ms). This reduction is driven by two factors: first, a coarser granularity amortizes fixed PCIe transaction overhead; second, larger chunks reduce the total data volume transferred by allowing longer "tail" segments to remain on the GPU without being persisted to the host. However, this I/O saving comes at the cost of increased *Comp Time* (12.48 ms → 17.93 ms), a phenomenon governed by the *persistence threshold*. Since MTServe only backs up KV states to the CPU tier once they accumulate into a full chunk, any un-offloaded tail tokens are lost if the user is evicted from the GPU. Consequently, these missing segments must be re-encoded from scratch during the next visit to reconstruct the sequence context, thereby inflating the computational burden. We select **1024** as our default configuration. At this granularity, MTServe captures the steepest drop in I/O wait time while minimizing the extra computational burden associated with larger chunks, yielding the optimal end-to-end performance.

6 Related Work

Generative Recommendation (GR). The paradigm of recommendation systems has significantly evolved in recent years. While deep learning-based methods [3, 5, 12, 26, 31, 32, 41] dominated for nearly a decade, they often face limitations in scalability and expressive power. The rise of large language models has ignited a shift towards generative recommendation [20, 28, 37]. This novel paradigm re-frames recommendation as a sequence transduction problem, leveraging their deep semantic understanding and open-ended generation capability to directly produce recommendations, explanations, or personalized content, thereby aiming for more expressive, unified, and user-centric systems.

Systems for GR. Recently, numerous generative recommendation systems have emerged in the industry, covering both training and inference optimizations [2, 6–9, 13, 15, 16, 18, 19, 22, 24, 25, 30, 33–36, 40]. For instance, OneRec [7] focuses on single-stage generative recommendation by adopting an MoE architecture and an Iterative Preference Alignment strategy. MTGenRec [33] leverages dynamic embedding tables, automated table merging, and two-stage ID deduplication to enhance the scalability of industrial models. Unlike these previous systems, MTServe identifies the unique optimization potential in the user-side KV cache and employs hierarchical storage and asynchronous pipelining to effectively improve inference performance.

LLM Inference Systems. LLM serving engines have achieved remarkable efficiency through fine-grained KV cache management [10, 14, 17, 21, 23, 27, 39]. Systems like vLLM [17], TensorRT-LLM [23], and SGLang [39] optimize serving via paged cache, dynamic prefix reuse, and reactive CPU swapping. However, their prefix-centric designs and reactive eviction policies are inherently tailored for volatile chat sessions. Under GR's massive, individualized cache churn, their reliance on blocking, page-level swapping triggers severe synchronization stalls. To further expand capacity, concurrent work like LMCache [21] advances hierarchical caching with chunk-level batched transfers. While effective for capacity scaling, it remains deeply anchored to standard LLM semantics. When confronted with GR's dominant access pattern—massive, concurrent per-user state restorations—their general-purpose execution pipelines struggle to mask the resulting I/O bursts. Ultimately, existing hierarchical frameworks lack the tailored, fine-grained compute-transfer coordination required to satisfy the stringent latency constraints of industrial GR serving.

7 Conclusion

This paper presented MTServe, a hierarchical KV cache system that overcomes the GPU memory wall for reusing the KV caches in serving generative recommendation models. MTServe achieves high efficiency with extensive system optimizations including a dual-granularity (Page-Chunk) storage layout, proactive cache offloading, and an asynchronous multi-stream pipeline. Experimental results demonstrate that MTServe accelerates end-to-end model inference by up to 3.5×.

Moving forward, we will explore three interesting directions: (i) applying compression techniques to the KV caches to improve the effective storage capacity and reduce data movement costs; (ii) deepening the storage hierarchy to include local NVMe SSDs to enlarge cache size and store cold users; (iii) enabling cross-machine transfer of the KV caches for scheduling and load balance.

References

[1] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. *arXiv preprint arXiv:2409.12740* (2024).

[2] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang, Xiang Wang, and Tat-Seng Chua. 2024. On Softmax Direct Preference Optimization for Recommendation. *Advances in Neural Information Processing Systems* 37 (2024), 27463–27489.

[3] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*. 7–10.

[4] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffenetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. 2024. LLM-Inference-Bench: Inference Benchmarking of Large Language Models on AI Accelerators. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1362–1379.

[5] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*. 191–198.

[6] Sunhao Dai, Jiakai Tang, Jiahua Wu, Kun Wang, Yuxuan Zhu, Bingjun Chen, et al. 2025. Onepiece: Bringing Context Engineering and Reasoning to Industrial Cascade Ranking System. *arXiv preprint arXiv:2509.18091* (2025).

[7] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *arXiv preprint arXiv:2502.18965* (2025).

[8] Yijie Ding, Jiacheng Li, Julian McAuley, and Yupeng Hou. 2026. Inductive Generative Recommendation via Retrieval-based Speculation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 14675–14683.

[9] Yue Dong, Han Li, Shen Li, Nikhil Patel, Xing Liu, Xiaodong Wang, and Chuanhao Zhuge. 2025. Scaling Generative Recommendations with Context Parallelism on Hierarchical Sequential Transducers. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 1058–1061.

[10] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. arXiv:2403.19708 [cs.CL] https://arxiv.org/abs/2403.19708

[11] Chongming Gao, Shijun Li, Yuan Zhang, Jiawei Chen, Biao Li, Wenqiang Lei, Peng Jiang, and Xiangnan He. 2022. KuaiRand: An Unbiased Sequential Recommendation Dataset with Randomly Exposed Videos. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. Association for Computing Machinery, New York, NY, USA, 3953–3957. https://doi.org/10.1145/3511808.3557624

[12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *26th International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence, 1725–1731.

[13] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, et al. 2025. MTGR: Industrial-scale Generative Recommendation Framework in Meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5731–5738.

[14] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, and Yuxiong He. 2024. DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference. arXiv:2401.08671 [cs.PF] https://arxiv.org/abs/2401.08671

[15] Yanhua Huang, Yuqi Chen, Xiong Cao, Rui Yang, Mingliang Qi, Yinghao Zhu, Qingchang Han, Yaowei Liu, Zhaoyu Liu, Xuefeng Yao, et al. 2025. Towards Large-scale Generative Ranking. *arXiv preprint arXiv:2505.04180* (2025).

[16] Tongyoung Kim, Soojin Yoon, Seongku Kang, Jinyoung Yeo, and Dongha Lee. 2024. SC-Rec: Enhancing Generative Retrieval with Self-consistent Reranking for Sequential Recommendation. *arXiv preprint arXiv:2408.08686* (2024).

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[18] Yaoyiran Li, Xiang Zhai, Moustafa Alzantot, Keyi Yu, Ivan Vulić, Anna Korhonen, and Mohamed Hammad. 2024. Calrec: Contrastive Alignment of Generative LLMs for Sequential Recommendation. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 422–432.

[19] Fake Lin, Binbin Hu, Zhi Zheng, Xi Zhu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, and Tong Xu. 2026. Token-level Collaborative Alignment for LLM-based Generative Recommendation. In *Proceedings of the ACM Web Conference 2026*. 6909–6919.

[20] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative Recommender with End-to-end Learnable Item Tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 729–739.

[21] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, et al. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).

[22] Xinchen Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. 2025. Qarm: Quantitative Alignment Multi-modal Recommendation at Kuaishou. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5915–5922.

[23] NVIDIA. 2023. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM.

[24] Fabian Paischer, Liu Yang, Linfeng Liu, Shuai Shao, Kaveh Hassani, Jiacheng Li, Ricky Chen, Zhang Gabriel Li, Xiaoli Gao, Wei Shao, et al. 2024. Preference Discerning with LLM-Enhanced Generative Retrieval. *arXiv preprint arXiv:2412.08604* (2024).

[25] Gustavo Penha, Ali Vardasbi, Enrico Palumbo, Marco De Nadai, and Hugues Bouchard. 2024. Bridging Search and Recommendation in Generative Retrieval: Does One Task Help the Other?. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 340–349.

[26] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online Learning of Social Representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 701–710.

[27] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Trans. Storage* (Nov. 2025). https://doi.org/10.1145/3773772 Just Accepted.

[28] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender Systems with Generative Retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[29] Jie Sun, Shaohang Wang, Zimo Zhang, Zhengyu Liu, Yunlong Xu, Peng Sun, Bo Zhao, Bingsheng He, Fei Wu, and Zeke Wang. 2026. Bat: Efficient Generative Recommender Serving with Bipartite Attention. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 223–238.

[30] Chunqi Wang, Bingchao Wu, Zheng Chen, Lei Shen, Bing Wang, and Xiaoyi Zeng. 2025. Scaling Transformers for Discriminative Recommendation via Generative Pretraining. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2893–2903.

[31] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-scale Commodity Embedding for e-commerce recommendation in alibaba. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 839–848.

[32] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD'17*. 1–7.

[33] Yuxiang Wang, Chi Ma, Xiao Yan, Mincong Huang, Xiaoguang Li, Ruidong Han, Bin Yin, Shangyu Chen, Xiang Li, Fei Jiang, et al. 2026. MTGenRec: An Efficient Distributed Training System for Generative Recommendation Models in Meituan. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2482–2493.

[34] Songpei Xu, Shijia Wang, Da Guo, Xianwen Guo, Qiang Xiao, Bin Huang, Guanlin Wu, and Chuanjiang Luo. 2025. Climber: Toward Efficient Scaling Laws for Large Recommendation Models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6193–6200.

[35] Liu Yang, Fabian Paischer, Kaveh Hassani, Jiacheng Li, Shuai Shao, Zhang Gabriel Li, Yun He, Xue Feng, Nima Noorshams, Sem Park, et al. 2024. Unifying Generative and Dense Retrieval for Sequential Recommendation. *arXiv preprint arXiv:2411.18814* (2024).

[36] Jun Yin, Zhengxin Zeng, Mingzheng Li, Hao Yan, Chaozhuo Li, Weihao Han, Jianjin Zhang, Ruochen Liu, Allen Sun, Denvy Deng, et al. 2024. Unleash LLMs Potential for Recommendation by Coordinating Twin-tower Dynamic Semantic Token Generator. *arXiv preprint arXiv:2409.09253* (2024).

[37] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 58484–58509.

[38] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2026. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. In *Proceedings of the ACM Web Conference 2026*. 8162–8170.

[39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems* 37 (2024), 62557–62583.

[40] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. 2025. OneRec-v2 Technical Report. *arXiv preprint arXiv:2508.20900* (2025).

[41] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep Interest Evolution Network for Click-through Rate Prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 5941–5948.

A Dataset Details and Statistics

To comprehensively evaluate MTServe, we utilize two datasets with distinct sequence length distributions. Table 4 summarizes the key statistics, including user count, total inference requests, and sequence length characteristics.

Table 4: Statistics of the evaluation datasets.

Dataset	#Users	#Reqs.	Min Seq.	Max Seq.	Avg Seq.
KuaiRand-1K	1,000	1,181,699	1	20,000	6,375
MT Dataset	2,884	2,146,179	4,000	6,000	5,251

KuaiRand-1K Preprocessing. As a public benchmark from the video-sharing platform Kuaishou, this dataset exhibits a high variance in user interaction histories (ranging from 1 to 20,000). To simulate a realistic streaming inference workload, we organize user behaviors chronologically by timestamp and segment them into individual requests using 60-second intervals. This method preserves the natural temporal arrival patterns and cache churn characteristic of online services, allowing us to evaluate the system's adaptability.

MT Dataset Preprocessing. The MT dataset is a production trace sourced from Meituan. We specifically filter this dataset to isolate users with exceptionally long interaction histories (between 4,000 and 6,000). By eliminating short-sequence noise, this dataset provides a stable, high-intensity environment to assess MTServe's capacity backpressure and hardware robustness under sustained VRAM memory pressure.

B Additional Experimental Results

B.1 Sensitivity Analysis of GPU Cache Capacity

Table 5 provides a concise comparison of the system's performance under various GPU cache capacities. We observe a clear trade-off: while increasing the VRAM footprint to 30 GB minimizes latency to 31.5 ms, the system remains highly effective even at a 10 GB limit, highlighting the flexibility of our hierarchical storage design.

Table 5: End-to-end latency across different GPU Primary Cache capacities (BS=8). Reference *Recompute* latency: 143.6 ms.

Primary Cache (#Blocks)	VRAM (approx.)	Total Latency (ms)
20,480	10 GB	**76.5**
30,720	15 GB	**52.1**
40,960	20 GB	**40.2**
51,200	25 GB	**34.9**
61,440	30 GB	**31.5**

C Pseudocode

Algorithm 1 illustrates the inference pipeline of the generative recommendation model integrated with our hierarchical KV cache manager. Specifically, the pipeline initiates metadata preparation and KV onloading in the background (Lines 1–2) while concurrently processing input embeddings (Lines 3–5). Metadata tasks, such as page allocation and LRU-driven eviction, are offloaded to the CPU to avoid stalling the main inference thread. Synchronization is performed implicitly within the transformer layers (Line 8.3) to ensure data availability before attention computation, allowing the transfer of deeper KV states to overlap with the computation of initial layers. Finally, the system performs a non-blocking asynchronous offload of the updated cache (Line 9), ensuring state persistence without extending the request latency.

D Implementation Details

D.1 Hierarchical Indexing Mechanism

To bridge the dual storage granularities (Page and Chunk) introduced in Section 4.1, MTServe maintains a robust indexing architecture that decouples the model's logical view of user history from its physical locations.

- **User-to-Unit Mapping:** We maintain a logical view for each user. On the GPU, a user maps to a list of non-contiguous *Logical Page IDs*; on the CPU, a user maps to a list of *Logical Chunk IDs*. This decoupling allows the logical sequence to appear contiguous to the attention mechanism while being physically scattered.
- **Page Table Translation:** A global page table translates these logical IDs into physical addresses. For the GPU Tier, it resolves a Logical Page ID to a physical index in the paged tensor; for the CPU Tier, it resolves a Chunk ID to a memory pointer. This table serves as the atomic unit for all memory operations, including allocation and LRU-based reclamation.
- **Layer-Centric Layout:** MTServe organizes the physical KV cache into a layer-first layout rather than a monolithic token-centric pool. This structural isolation inherently enables layer-wise compute-transfer overlapping, allowing the system to asynchronously fetch KV chunks for layer $l + 1$ while concurrently computing the attention for layer l, thereby effectively hiding I/O overhead.

D.2 Hardware Environment and Implementation

All experiments are conducted on a high-performance server node equipped with an NVIDIA A100 GPU (80 GB HBM2e, SM 8.0, PCIe Gen4 x16). The host side features 2× AMD EPYC 7713 CPUs (128 cores in total) and 928 GB of DDR4 RAM, providing abundant capacity to serve as the scalable backup store for our hierarchical KV cache. The cross-platform validation runs on NVIDIA H20-3e GPUs (140 GB HBM3, SM 9.0, PCIe Gen5 x16) with 2× Intel Xeon Platinum 8480+ CPUs (112 cores in total) and 1,568 GB of host memory. Experiments run on Linux kernels 4.18 (primary platform) and 5.10 (H20 platform) with CUDA driver 535.129.03 / 550.127.08 respectively. MTServe is fundamentally implemented using PyTorch. To minimize software-side overhead and saturate hardware bandwidth, critical performance-sensitive components—such as the scatter/gather memory layout transformations and the double-buffered metadata management pipeline—are implemented as custom CUDA and C++ extensions.

Algorithm 1 Overall HSTU Inference Pipeline using Asynchronous Paged KV Cache Management.

Require: *Input Batch*: a batch of user and feature IDs;
AsyncKVCacheManager: a manager for offloading/onloading KVs, page allocation, and LRU eviction.
Ensure: *Output Logits*: final ranking scores for all users in the batch.

1. Fetch Cache Lengths: Retrieve users' total historical sequence length.
  $TotalHistLengths \leftarrow$ AsyncKVCacheManager.get_total_cache_length($InputBatch.users$)
2. Asynchronously Prepare Metadata and Onload KVs: Initiate background page allocation, LRU ranking, and *asynchronous* transfer of historical KVs from host to GPU onload buffer.
  $MetadataFut, OnloadFut \leftarrow$ AsyncKVCacheManager.SubmitPrepareMetadataAndOnloadAsync(TotalHistLengths, batch_size)
3. Preprocess Input Tokens: Strip cached history tokens from inputs.
  $StrippedBatch \leftarrow$ AsyncKVCacheManager.strip_cached_tokens($InputBatch$)
4. Compute Embeddings: Generate embeddings for non-cached inputs.
  $Embeddings \leftarrow$ EmbeddingCollection($StrippedBatch.features$)
5. Transform Data Layout: Convert embeddings into JaggedArray for HSTU input.
  $JaggedData \leftarrow$ HSTUBlock.preprocessor($Embeddings, StrippedBatch$)
6. Await Metadata (Blocking): Wait until metadata preparation (page allocation, LRU) is fully complete. *(Note: Onload operation proceeds asynchronously in the background.)*
  $KVCacheMetadata \leftarrow$ AsyncKVCacheManager.AwaitMetadata($MetadataFut$)
7. Update Metadata and Commit: Update metadata to include lengths of the newly arrived batch sequence and commit the onloaded data from the buffer into the main paged cache.
  $KVCacheMetadata.total\_history\_offsets$ += $JaggedData.num\_offsets$
  $KVCacheMetadata.total\_history\_lengths$ += $JaggedData.num\_candidates$
  AsyncKVCacheManager.CommitOnloadBufferToCache($InputBatch.users$)
8. HSTU Core Computation Loop: Execute Attention computation for each Transformer layer l.
for each layer l in HSTU model **do**
    8.1. Compute and Reshape QKV: $Q, K, V \leftarrow$ ComputeQKV($JaggedData.values$)
    8.2. Append New KVs to Paged Cache: Store K, V for the current input into the Paged KV Cache.
      append_kvcache(K, V, *KVCacheMetadata*.position, *KVCacheMetadata*.offsets,
                KVCacheMetadata.indices/indptr/last_page_lens)
    8.3. Implicit Sync for Historical KVs (Critical): Implicitly wait for the historical KV data specific to layer l to be fully loaded from the onload buffer into the paged cache. This synchronization is managed by the layer's *KVOnloadHandle*.
    8.4. Compute Attention Scores: $OutputValues \leftarrow$ HSTUAttention(Q, K, V)
    8.5. Fuse and Feed-Forward: $JaggedData.values \leftarrow$ PostAttentionProcess($OutputValues$)
end for
9. Asynchronously Offload KVs: Background save the updated KV cache to host memory.
  AsyncKVCacheManager.SubmitOffloadAsync($KVCacheMetadata$)
10. Final Outputs: Finalize outputs and compute final ranking logits.
  $HSTUOut \leftarrow$ HSTUBlock.postprocessor($JaggedData.values$)
  $FinalLogits \leftarrow$ MLP($HSTUOut$)
return $OutputLogits : (FinalLogits)$

D.3 Adaptation of Baselines

To ensure a fair and rigorous evaluation, we augmented the baseline systems with critical hardware-level optimizations. This ensures that the performance gaps reported in Section 5 stem fundamentally from architectural scheduling differences rather than missing basic engineering implementations.

TensorRT-LLM. We co-designed a hierarchical GPU-CPU variant of *TensorRT-LLM* with NVIDIA engineers. In its native orchestration, TensorRT-LLM relies on single-granularity page management. However, under generative recommendation workloads, running this baseline at a uniform fine granularity (e.g., a 32-token page) creates excessive discrete PCIe transactions. This severe I/O bottleneck degrades the end-to-end latency to **239 ms** on the KuaiRand-1K dataset at a batch size of 8. To eliminate this inefficiency, we augmented TensorRT-LLM with our *Page–Chunk dual-granularity storage layout* alongside a custom *double buffering mechanism*.

LMCache. Similarly, we integrated the *double buffering mechanism* into the *LMCache* baseline to help it mask basic memory allocation and PCIe transfer overheads. Without this enhancement, LMCache suffers from severe synchronization stalls. For instance, on the KuaiRand-1K dataset at a batch size of 4, the native LMCache without double buffering yields an end-to-end latency of **51.0 ms**. By equipping it with our double buffering implementation, its latency drops significantly to **35.2 ms**.

By incorporating these physical layout and buffering optimizations into both baselines, we push their hardware utilization to the

limit. Therefore, the remaining superiority of MTServe demonstrated in our experiments is solely attributable to its holistic asynchronous pipeline and zero-copy compute-transfer overlap.

E Detailed Memory Footprint Analysis

This section details the GPU memory allocation for MTServe during active inference ($B = 8, L_{max} = 40,008$) using `bfloat16` precision. The total peak footprint of *41.67 GiB* (42,671 MiB), primarily driven by the persistent KV cache and the pre-allocated inference workbenches.

E.1 Persistent KV Cache Storage (~24.89 GiB)

This component represents the historical user state retained for cross-request reuse. It is pre-allocated as a unified `cache_table` tensor with the shape $[L, (N_p + N_o), 2, S_{page}, H, D]$, where $L = 8$, $S_{page} = 32$, $H = 4$, and $D = 128$. The allocation comprises two functional regions:

- **Primary Cache (N_p):** A fixed pool of 40,960 pages reserved for active users.
- **Onload Buffer (N_o):** The staging area capacity is determined by the maximum tokens per batch to ensure all incoming data can be accommodated: $N_o = \lceil (B \times L_{max})/S_{page} \rceil = \lceil 320,064/32 \rceil \approx 10,008$ pages.

The total allocation for these 50,968 pages ($N_p + N_o$) occupies approximately **25,484 MiB** ($8 \times 50,968 \times 2 \times 32 \times 4 \times 128 \times 2$ bytes).

E.2 Inference Workbench (~14.65 GiB)

To eliminate dynamic memory management overhead, each of the 8 Transformer layers pre-allocates a dedicated "workbench" for intermediate activations. The size is governed by the peak token capacity $T_{max} = B \times L_{max} = 320,064$. Each layer reserves approximately **1,875 MiB**, broken down as follows:

- **UVQK Projection Buffer (1,250 MiB):** Stores the intermediate U, V, Q, K projections. With a hidden dimension of 128 per head and 4 heads, this requires $T_{max} \times (128 \times 4 \times 4) \times 2 \approx 1,250$ MiB.
- **Output Buffer (625 MiB):** Stores the activation outputs of the attention and FFN blocks, requiring $T_{max} \times 1024 \times 2 \approx 625$ MiB.

Across all 8 layers, this workbench occupies a total of **15,000 MiB**.

F Limitations and Future Work

Limitations. First, the current two-tier GPU–CPU hierarchy relies on PCIe bandwidth for data movement, which may become a bottleneck under extreme scale. Second, MTServe currently operates within a single-node scenario and does not address cross-node KV cache migration for distributed load balancing.

Future Work. Moving forward, we plan to explore three directions. (i) *KV cache compression:* Applying quantization or distillation techniques to KV states to improve effective storage capacity and reduce PCIe data movement costs, similar to recent advances in LLM KV cache compression. (ii) *Deeper storage hierarchy:* Integrating local NVMe SSDs as a third tier to store cold user states, enabling virtually unlimited cache capacity at the cost of higher access latency for rarely-active users. (iii) *Distributed KV cache transfer:* Enabling cross-machine migration of KV caches via RDMA, which allows dynamic scheduling and load balancing across a cluster of serving nodes.